
Output Traces Do Not Identify Decision-Relevant Hidden State in LLM Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Output traces do not identify whether hidden state matters for action. Two agent
2 systems can produce identical observable outputs while the influence of hidden
3 state on behavior ranges from zero to maximal. We prove this Output-Trace Identifi-
4 fiability Gap (Theorem 1) and establish two practical certificates for narrowing it:
5 a structural certificate that bounds how much hidden information the system could
6 contain, and a behavioral certificate that measures how much hidden information
7 actually affects decisions. We demonstrate both certificates across three agent
8 architectures—a ReAct-style agent where hidden scratchpad state is largely dor-
9 mant, a diffusion language model where decision relevance concentrates at the final
10 binding step, and a multi-agent system where a private communication channel
11 carries near-maximal decision influence. Our framework separates the structural
12 question (how much hidden state *could* exist?) from the behavioral question (does
13 it *affect* decisions?), giving audit practitioners a principled alternative to black-box
14 opacity.

15 1 Introduction

16 Consider a ReAct-style LLM agent deployed with an internal scratchpad that is not recorded in the
17 deployment log. A regression test on output traces shows no behavioral change. Controlled replay
18 of the same trace with the scratchpad disabled, however, reveals a 0.0163-bit shift in the tool-token
19 probability distribution on planning tasks. For a safety auditor, this raises an immediate concern:
20 could the missing scratchpad covertly influence the agent’s tool choices without leaving a trace?

21 The disagreement between output-only and replay-based audit is not a probing artifact. It is a
22 structural feature of the audit: the recorded trace cannot distinguish dormant hidden capacity from
23 active hidden capacity. This is actually two questions, not one—how much hidden state is structurally
24 possible, and how much of it actually affects behavior.

25 This failure is not a limitation of a particular probe or estimator. It is an identifiability gap. Given any
26 observable joint distribution over traces and actions, there exist two agent realizations—one with zero
27 residual decision relevance, one with maximal—that produce exactly the same observational data.

28 1.1 Intuitive Overview

29 The paper’s core idea can be stated without formalism. An auditor observes the outputs of an agent—
30 the tokens it generates, the tools it calls—but not everything inside the agent: internal buffers, cached
31 activations, intermediate reasoning states. The question is whether this unobserved internal state
32 could change the agent’s decisions.

33 This is actually two separate questions:

1. **Capacity:** How much information could the hidden state carry? A large internal buffer could theoretically encode a lot of information, even if it is never used for decisions.
2. **Usage:** Does the hidden state actually affect decisions? It might be structurally present but behaviorally irrelevant—like a whiteboard that is never written on.

Standard audits cannot tell these apart. Two systems with the same observable outputs could have very different internal dynamics: one where hidden state is completely passive, and one where it drives every action. This is an *identifiability gap*, not a measurement limitation.

We provide two certificates. The *static certificate* answers the capacity question by examining the system’s deployment topology—which internal channels are logged and which are not—and computing an upper bound on how much hidden information could be present. The *dynamic certificate* answers the usage question by running controlled experiments—replaying traces with and without hidden state, or perturbing suspected internal channels—to measure whether hidden state actually changes the output distribution.

The experiments show that both certificates are needed: the same system can have large hidden capacity that is nearly unused (ReAct scratchpad), or moderate capacity that is highly used at critical moments (LLaDA denoising, multi-agent peer reports).

Contributions.

1. **Theorem 1—Output-Trace Identifiability Gap.** Behavioral equivalence—identical observable output distributions—does not imply audit equivalence: two systems with the same recorded traces can have arbitrarily different amounts of hidden state influence on actions.
2. **Dual certificates for closing the gap.** A static certificate that upper-bounds hidden-state capacity from deployment topology (Proposition 1), and a dynamic certificate that lower-bounds hidden state’s actual decision influence from controlled probes (Proposition 2). The two answers address qualitatively different audit questions.
3. **Empirical demonstration on three architectures.** We instantiate both certificates on a ReAct agent (near-zero hidden influence), a diffusion LM (mid-spectrum influence at the final binding step), and a multi-agent system (near-saturation influence through an unlogged peer-report channel).
4. **Formal supplementary proofs.** All theoretical claims are mechanically verified; see Appendix D.

Paper structure. Section 2 discusses related work. Section 3 formalizes the agent setup and defines the two audit quantities. Section 4 proves the identifiability gap (Theorem 1). Section 5 develops the two gap-closing certificates. Section 6 demonstrates them on three architectures. Section 7 discusses limitations and future directions. Formal proofs are in Appendix D; experimental protocols are in Appendix B; synthetic ground-truth validation is in Appendix C.

2 Related Work

Probing, patching, and causal abstraction. Causal abstraction [Geiger et al., 2021], latent-knowledge elicitation [Burns et al., 2022], nullspace projection [Ravfogel et al., 2020], and amnesic probing [Elazar et al., 2021] give proxy-style evidence: when they expose a probe variable $Z_t = \varphi(S_t)$ and the required conditional assumptions hold, $I(Z_t; A_t | \bar{T}_t)$ is a proxy certificate. Causal tracing/ROME [Meng et al., 2022], representation engineering [Zou et al., 2023], and activation addition [Turner et al., 2023] give intervention evidence when the patch affects action only through S_t .

Black-box audit and network information theory. Casper et al. [Casper et al., 2024] argue that black-box access is insufficient for auditing internal agent mechanisms. Our work makes this insufficiency quantitative for δ_{act} : output-only access cannot distinguish $\delta_{\text{act}} = 0$ from $\delta_{\text{act}} = H(A_t | \bar{T}_t)$. Pearl’s partial-identification framework [Pearl, 2009] applies: latent-variable models are generally not identifiable from observational data alone. We pair this principle with explicit gap-closers: $\varepsilon_{\text{state}}^{\text{UB}}$ from deployment topology when structural access is available, and $\delta_{\text{act}}^{\text{LB}}$ via admissible

83 probes. The static certificate applies a cut-set upper bound [El Gamal and Kim, 2011] to the
84 time-unrolled DAG (full derivation in Appendix A).

85 **Diffusion language-model agents.** LLaDA is a large-scale masked diffusion language model with
86 bidirectional denoising and instruction-following ability [Nie et al., 2025]. Recent work studies
87 diffusion LMs in multi-step decision and tool-use workflows, including matched comparisons to
88 autoregressive agents [Zhen et al., 2026]. Intermediate denoising latents make these systems a natural
89 non-ReAct hidden channel for the dynamic certificate.

90 3 Setup and Realization Formalism

91 3.1 Intuition: What Are We Modeling?

92 Consider an LLM agent responding to a user query. Internally, the model computes intermediate
93 representations—attention patterns, hidden states, cache entries. Some of these are captured in the
94 deployment log (the *recorded trace*), others are not (the *unrecorded trace*). An auditor can inspect
95 the recorded trace and the agent’s final outputs, but cannot directly observe the unrecorded internals.

96 The core modeling question is: given what we can see (recorded traces and actions), what can we
97 conclude about what we cannot see (unrecorded state)? Our framework makes this precise with four
98 random variables:

- 99 • T_t : the agent’s full internal computation at step t (inaccessible).
- 100 • $\tilde{T}_t \subseteq T_t$: the subset recorded in the deployment log (auditor-visible).
- 101 • S_t : the operative state—the specific internal representation that drives action.
- 102 • A_t : the agent’s observable output (next token or tool call).

103 The auditor’s goal is to determine whether information in $T_t \setminus \tilde{T}_t$ (the gap between full and recorded
104 trace) could affect—or does affect—the action A_t .

105 Standard information-theoretic notation follows Cover and Thomas [2006]. All logarithms are base
106 2; entropy and mutual information are reported in bits.

107 At step t : *full trace* T_t (all intermediate activations), *recorded trace* $\tilde{T}_t \subseteq T_t$ (auditor-visible),
108 *operative state* S_t (internal computation), *action* A_t (next token), *unrecorded sources* U_t (exogenous
109 inputs not in $\tilde{T}_t$).

Symbol	Definition
S_t	Operative state (internal computation at step t)
T_t	Full trace (all intermediate activations)
$\tilde{T}_t$	Recorded trace (auditor-visible subset of T_t)
A_t	Action (next observable token or tool call)
U_t	Unrecorded sources (exogenous inputs not in $\tilde{T}_t$)
$\varepsilon_{\text{state}}^{\text{UB}}$	Static certificate: upper bound on $H(S_t \tilde{T}_t)$
$\delta_{\text{act}}^{\text{LB}}$	Dynamic certificate: lower bound on $I(S_t; A_t \tilde{T}_t)$

Table 1: Summary of notation used throughout the paper. The two core audit quantities are $\varepsilon_{\text{state}}^{\text{UB}}$ (hidden-state uncertainty bound) and $\delta_{\text{act}}^{\text{LB}}$ (decision relevance lower bound).

110 **Definition 1** (Agent Realization). An agent realization $\mathcal{R} = (P_S, f_T, f_A, \pi_U)$ consists of an operative-
111 state distribution P_S over S_t , a trace-generation map $f_T : S_t \rightarrow \Delta(\tilde{T}_t)$, an action policy $f_A : (S_t, \tilde{T}_t) \rightarrow \Delta(A_t)$, and a source policy π_U governing U_t . The realization induces an observational
112 marginal $P_{\mathcal{R}}(\tilde{T}_t, A_t)$ on the support of a fixed deployment prompt distribution P_{prompt} . In other
113 words, an agent realization is a specification of how hidden state, recorded traces, and actions relate
114 probabilistically. Different realizations can produce the same observable data—this is where the
115 identifiability problem arises.
116

117 **Definition 2** (Behavioral Equivalence). *Two realizations $\mathcal{R}_0, \mathcal{R}_1$ are behaviorally equivalent, written*
 118 *$\mathcal{R}_0 \sim_B \mathcal{R}_1$, iff they produce identical joint distributions over observable variables on the deployment*
 119 *prompt distribution:*

$$P_{\mathcal{R}_0}(\tilde{T}_t, A_t) = P_{\mathcal{R}_1}(\tilde{T}_t, A_t) \quad \forall (\tilde{T}_t, A_t) \in \text{supp}(P_{\text{prompt}}).$$

120 *Behavioral equivalence concerns the fixed deployment marginal only. Paraphrase variation and red-*
 121 *teaming are generalization probes that explore the policy under distribution shift; they are not audits*
 122 *on the deployment marginal and do not by themselves refute behavioral equivalence. Concretely,*
 123 *behavioral equivalence means that an auditor who only looks at recorded traces and tool calls cannot*
 124 *tell two realizations apart. The identifiability gap exists because this observational equivalence does*
 125 *not guarantee that hidden state is equally influential across both realizations.*

126 3.2 What We Are Trying to Measure

127 **Definition 3** (Core Audit Quantities). *We target two quantities that capture distinct aspects of*
 128 *hidden-state risk:*

- 129 • **Hidden-state uncertainty** ($\varepsilon_{\text{state}} := H(S_t \mid \tilde{T}_t)$): *How much information about the agent's*
 130 *internal state remains unknown even after seeing the recorded trace. This answers: how*
 131 *much hidden information could there be?*
- 132 • **Decision relevance** ($\delta_{\text{act}} := I(S_t; A_t \mid \tilde{T}_t)$): *How much of that hidden information actually*
 133 *influences the agent's next action. This answers: does the hidden information affect*
 134 *behavior?*

135 *The two quantities are linked by the data processing inequality ($\delta_{\text{act}} \leq \varepsilon_{\text{state}}$): decision-relevant*
 136 *hidden state is always a subset of total hidden state. The gap between them is what makes auditing*
 137 *nontrivial. Section 5 gives an upper bound for the first quantity and a lower bound for the second.*

138 Readers who prefer a concrete example before the formal development may skip ahead to Section 6,
 139 where both certificates are instantiated on real systems, and refer back to the definitions here as
 140 needed.

141 4 Output-Trace Identifiability Gap

142 The identifiability gap can be understood through a simple construction. Suppose an auditor observes
 143 a distribution over traces and actions $Q(T, A)$. There exist two distinct internal mechanisms consistent
 144 with Q : one where internal state is completely passive ($\delta_{\text{act}} = 0$) and one where internal state encodes
 145 the action itself ($\delta_{\text{act}} = H(A \mid T)$, maximal). Since both produce exactly the same Q , the observer
 146 cannot tell which case holds. Theorem 1 formalizes this.

147 **Theorem 1** (Output-Trace Identifiability Gap). *For notation simplicity in this general PMF setup, let*
 148 *$T := \tilde{T}_t$, $A := A_t$, $S := S_t$ (as defined in §3). Let S, T, A be finite. For any finite PMF Q on $T \times A$*
 149 *with $\sum_a Q(t, a) > 0$ for all $t \in \text{supp}(Q_T)$ and $|S| \geq |A|$, there exist two finite joint PMFs P_0, P_1*
 150 *on $S \times T \times A$ such that*

- 151 (i) P_0 and P_1 project to the same observational marginal Q : $\sum_s P_0(s, t, a) =$
 152 $\sum_s P_1(s, t, a) = Q(t, a)$;
- 153 (ii) under the audit functional $\delta_{\text{act}}(P) := I_P(S; A \mid T)$, we have $\delta_{\text{act}}(P_0) = 0$ and $\delta_{\text{act}}(P_1) =$
 154 $H_Q(A \mid T)$.

155 Hence no functional of the observational marginal Q alone can distinguish a realization with $\delta_{\text{act}} = 0$
 156 from one with maximal δ_{act} .

157 **Construction.** Define two PMFs on $S \times T \times A$:

$$\begin{aligned} P_0(s, t, a) &= \mathbf{1}[s = s_0] \cdot Q(t, a) \quad (\text{Dirac state: } S \text{ constant, no residual decision relevance}), \\ P_1(s, t, a) &= \mathbf{1}[s = \phi(a)] \cdot Q(t, a) \quad (\text{state copies action via injection } \phi : A \hookrightarrow S). \end{aligned}$$

158 For (i): $\sum_s P_0(s, t, a) = Q(t, a)$ and $\sum_s P_1(s, t, a) = Q(t, a)$ by construction—both project to the
 159 same Q . For (ii): In P_0 , $S = s_0$ deterministically, so $I_{P_0}(S; A | T) = 0$. In P_1 , $S = \phi(A)$, so A
 160 is determined by S given any T , giving $I_{P_1}(S; A | T) = H_Q(A | T)$. The cardinality condition
 161 $|S| \geq |A|$ guarantees the injection ϕ exists. $\square$

162 5 Gap-Closing Certificates

163 The data processing inequality gives $\delta_{\text{act}} \leq \varepsilon_{\text{state}}$. The certificates inherit that ordering: if $\varepsilon_{\text{state}}^{\text{UB}} = 0$,
 164 then $\delta_{\text{act}}^{\text{LB}} = 0$. The converse fails; $\delta_{\text{act}}^{\text{LB}} = 0$ remains compatible with large $\varepsilon_{\text{state}}^{\text{UB}}$. Each certificate
 165 uses a different access mode: structural access to deployment topology for $\varepsilon_{\text{state}}^{\text{UB}}$ (§5.1), controlled
 166 replay under wild/replay conditions, or proxy/intervention access to $Z_t = f(S_t)$ or hidden modules
 167 for conditional-DPI lower bounds (§5.2). The empirical core instantiates these access modes in a
 168 controlled ReAct agent.¹

169 5.1 Static Certificate: Upper Bound from Deployment Topology

170 The static certificate upper-bounds $\varepsilon_{\text{state}} = H(S_t | \tilde{T}_t)$ from deployment topology. Its value depends
 171 on *which components the auditor trusts*.

172 **Deployment topology and trust.** The auditor partitions system components into Trusted (protocol-
 173 budgeted), Untrusted (adversarial-budgeted), and Suspect (the audit target). Each macro-edge expands
 174 into a time-unrolled DAG $G_t = (V_t, E_t)$. Per-edge budgets depend on trust status: protocol budgets
 175 c_e^{proto} for Trusted edges (read from interface specifications), and adversarial budgets c_e^{adv} for Untrusted
 176 edges (worst-case capacity under the deployment format, by definition $c_e^{\text{adv}} \geq c_e^{\text{proto}}$). Appendix A
 177 gives the full budget taxonomy, software orthogonality (Lemma 1), and the non-orthogonal case.

178 The default-deny rule is conservative: absent cryptographic or organizational trust anchors, classify
 179 components as Untrusted. This includes cases where an untrusted component can encode extra
 180 state in otherwise protocol-compliant natural-language outputs, such as compressed rationales or
 181 steganographic coordination signals.

182 **Proposition 1** (Structural-Access Closer: Static Certificate via Cut-Set Bound). *For any cut Ω*
 183 *separating U_t from S_t with induced capacity $C_{\text{cut}}(\Omega) = \sup_{p(X_\Omega | \tilde{T}_t)} I(X_\Omega \rightarrow Y_{\Omega^c} | \tilde{T}_t, X_{\Omega^c})$,*

$$H(S_t | \tilde{T}_t) \leq \underbrace{H(S_t | T_t)}_{\varepsilon_{\text{nominal}}} + \min_{\Omega} C_{\text{cut}}(\Omega).$$

184 *Setting $\varepsilon_{\text{state}}^{\text{UB}} := \varepsilon_{\text{nominal}} + \min_{\Omega} C_{\text{cut}}(\Omega)$ gives $\varepsilon_{\text{state}} \leq \varepsilon_{\text{state}}^{\text{UB}}$.*

185 Here X_Ω denotes the input signals entering edges crossing the cut, Y_{Ω^c} the corresponding output
 186 signals, and X_{Ω^c} the boundary state on the sink side (formal definitions in Appendix A).

187 **Corollary 1** (Additive form). *Under software orthogonality, $\varepsilon_{\text{state}}^{\text{UB}} = \varepsilon_{\text{nominal}} + \min_{C \in C_{\text{unlogged}}} \sum_{e \in C} c_e$,*
 188 *a discrete min-cut on the unlogged subgraph.*

189 *Proof sketch:* Chain-rule gives $H(S_t | \tilde{T}_t) = H(S_t | T_t) + I(S_t; M_t | \tilde{T}_t)$ where $M_t = T_t \setminus \tilde{T}_t$.
 190 Every path from U_t to S_t crosses a cut Ω , so M_t is d -separated from S_t by the cross-cut signals; DPI
 191 yields $I(S_t; M_t | \tilde{T}_t) \leq C_{\text{cut}}(\Omega)$. Minimizing over cuts and applying the additive decomposition
 192 (Lemma 1, Appendix A) completes the proof.

193 **Corollary 2** (Autoregressive zero-cut). *If the system is a strict autoregressive core and $\tilde{T}_t$ includes*
 194 *the full context window, then $C_{\text{unlogged}} = \emptyset$, every cut has zero induced capacity, and $\varepsilon_{\text{state}}^{\text{UB}} = 0$.*

¹For readers who prefer concrete referents: $\varepsilon_{\text{state}}$ measures “how much about the agent’s internals can the auditor not see?” and δ_{act} measures “how much of what the auditor cannot see affects what the agent does?” If there is no hidden information, there can be no hidden influence, but hidden information can exist without being used.

5.2 Dynamic Certificate: Lower Bound from Behavioral Probes

The gray-box-access closer targets $\delta_{\text{act}} := I(S_t; A_t \mid \tilde{T}_t)$. Since S_t is unobservable, we specify admissible probes whose measured variables satisfy $X_t \rightarrow S_t \rightarrow A_t$ given $\tilde{T}_t$. Conditional DPI establishes the bound: $I(X_t; A_t \mid \tilde{T}_t) \leq \delta_{\text{act}}$.

Proposition 2 (Gray-Box-Access Closer: Probe Certificates from Conditional DPI). *For any admissible probe variable X_t with $X_t \rightarrow S_t \rightarrow A_t$ given $\tilde{T}_t$, the probe measurement $I(X_t; A_t \mid \tilde{T}_t)$ is a valid lower-bound certificate for δ_{act} .*

Admissible probe: Replay. Let $R_t \in \{\text{wild}, \text{replay}\}$ indicate whether a missing state fragment is reconstructed under the same visible trace $\tilde{T}_t$ (controlled re-execution, not paraphrase variation). When R_t is uniformly distributed, the replay certificate takes the form of a Jensen–Shannon divergence:

$$I(R_t; A_t \mid \tilde{T}_t) = \text{JS}(P_{\text{wild}}, P_{\text{replay}} \mid \tilde{T}_t) \leq \delta_{\text{act}}, \quad (\text{Replay Lower Bound})$$

where JS is the Jensen–Shannon divergence with equal mixture weight—a concrete manifestation of the mutual information $I(R; A \mid \tilde{T}_t)$ under the uniform mixture $R \sim \text{Bernoulli}(1/2)$. Discrete action spaces admit direct empirical JS estimation over either realized actions or model-reported action probabilities; the experiments below state which is used.

Admissible probe: Intervention. Let ξ_{hidden} be an exogenous perturbation to a suspected unlogged module (cache, memory buffer, scratchpad). If $\xi_{\text{hidden}} \rightarrow S_t \rightarrow A_t$ given $\tilde{T}_t$, then $I(\xi_{\text{hidden}}; A_t \mid \tilde{T}_t) \leq \delta_{\text{act}}$. Discrete action spaces admit direct conditional action-distribution shift estimates, such as JS divergence. For continuous action spaces, the same certificate targets $I(\xi_{\text{hidden}}; A_t \mid \tilde{T}_t)$ through variational lower-bound estimators such as InfoNCE [van den Oord et al., 2018] or MINE [Belghazi et al., 2018]. Requires gray-box perturbation access; the most direct causal probe of the three.

Admissible probe: Proxy. Let $Z_t = f(S_t)$ be a readable coarsening of the operative state. Since $Z_t \leftarrow S_t \rightarrow A_t$ given $\tilde{T}_t$, conditional DPI yields $I(Z_t; A_t \mid \tilde{T}_t) \leq \delta_{\text{act}}$, estimated via the cross-entropy gap of predictors with and without Z_t .

Continuous actions and estimator error. When A_t is continuous, the proof obligation does not change—only the estimator does. The audit should report a conservative lower confidence bound $\max\{0, \hat{I}_{\text{InfoNCE/MINE}} - \text{err}_{N, \mathcal{F}}\}$, where $\text{err}_{N, \mathcal{F}}$ covers finite-sample uncertainty, critic-class error, and optimization instability. Sample splitting and negative controls play the same role that bootstrap CIs play for discrete actions. Our formalization (Appendix D) covers the finite-discrete case; continuous-action certificates are an empirical extension of Proposition 2, not a claim that continuous MI estimators are mechanized.

Probe aggregation. A single certificate class can miss some causal paths. Define $\delta_{\text{act}}^{\text{LB}} := \max\{I(R_t; A_t \mid \tilde{T}_t), I(\xi_{\text{hidden}}; A_t \mid \tilde{T}_t), I(Z_t; A_t \mid \tilde{T}_t)\}$. By monotonicity of the maximum, the aggregate remains a valid lower bound on δ_{act} .

Indexed activation profiles. The scalar certificate is the maximum over an indexed family of admissible probes. Keeping the index gives an *activation profile*

$$\Delta(j) := I(X_t^{(j)}; A_t \mid \tilde{T}_t),$$

where j may denote a scratchpad field, a denoising step, an activation layer, or a communication edge. The static certificate identifies where residual capacity can reside; the activation profile identifies where that capacity is behaviorally used. The experiments below keep this algebra fixed while changing the index: module in ReAct, time in the diffusion LM, and communication edge in the multi-agent setting.

6 Empirical Instances Across the Gap

Theorem 1 identifies the possible range: behaviorally equivalent realizations can sit at 0 or at $H(A_t \mid \tilde{T}_t)$ while preserving the same observational marginal. The experiments do not construct

Logging level	$\varepsilon_{\text{state}}^{\text{UB}}$ (bits)	Bottleneck
ReAct, output only	16,464	scratchpad_read (16,384) [†] + router_logits (80)
ReAct, +log router	16,384	scratchpad_read (16,384)
ReAct, +log scratchpad	0	none
ReAct, full instrumentation	0	none
LLaDA-8B-Instruct ($K=128$)	32,768	denoising interface
Multi-agent, all logged	0	none
Multi-agent, peer report unlogged	32,768	peer_report $\rightarrow S_t$

Table 2: Static certificate geometry. Adding ReAct logs monotonically removes residual cut capacity; the same min-cut analysis localizes capacity in a scratchpad module, denoising interface, or peer-report edge. The static certificate drops to zero once all high-capacity channels are logged, confirming that logging ablation closes the structural gap.

such paired realizations²; they test whether the certificates distinguish regimes along the resulting δ -spectrum. The three systems map onto the two certificates as follows:

- **ReAct (dormant capacity):** Large static certificate ($\varepsilon_{\text{state}}^{\text{UB}} = 16,384$ bits) but near-zero dynamic certificate. Hidden capacity exists structurally but is not used.
- **LLaDA (late binding):** Capacity exists throughout denoising, but decision relevance appears only at the final step. The activation profile reveals *when* hidden state matters.
- **Multi-agent (active channel):** The unlogged peer-report channel carries near-maximal decision relevance ($\delta_{\text{act}}^{\text{LB}} = 0.901$ bits out of 1.0 ceiling). Both certificates are needed to characterize the risk.

The static min-cut (§6.1) provides the shared topology coordinate; §§6.2–6.4 traverse the behavioral coordinate. Read-only proxy calibration is reported in Appendix B; synthetic ground-truth validation is reported in Appendix C. All pipelines ran on a single M4 Max workstation.

6.1 Static Certificate: Min-Cut and Logging Ablation

We extract the time-unrolled DAG from Qwen2.5-7B-Instruct ReAct execution traces on tool-selection tasks (Fig. 1, left). Per-edge budgets c_e are assigned from interface specifications (§5.1); the min-cut on unlogged edges gives $\varepsilon_{\text{state}}^{\text{UB}}$. For these deployments, $\varepsilon_{\text{nominal}} = H(S_t | T_t) \approx 0$ because the operative state is a deterministic function of the full execution trace (weights fix activations given the complete input); the reported min-cut therefore dominates the bound. Varying logging levels recomputes the min-cut; Table 2 reports the stepwise reduction from 16,464 to 0 bits. The dominant bottleneck is the scratchpad-read path (16,384 bits).

[†]The 16,384-bit term is the unlogged channel capacity of the scratchpad-read interface in this deployment spec: a 2,048-dimensional scratchpad state quantized at 8 bits per dimension, hence $2,048 \times 8 = 16,384$ bits. It is a worst-case capacity ceiling, not an estimate that all bits are behaviorally used.

6.2 Dynamic Causal Certificates: Intervention and Controlled Replay

This experiment separates the two audit axes. The topology, and therefore $\varepsilon_{\text{state}}^{\text{UB}}$, stays fixed; $\delta_{\text{act}}^{\text{LB}}$ varies with task. The unlogged scratchpad is present in both splits. It is irrelevant for calculator tasks and necessary for planning tasks. Intervention perturbs the scratchpad by masking or replacement; controlled replay removes it while holding $\tilde{T}_t$ fixed. Table 3 reports JS divergence. For the headline replay contrast, argmax tool counts are unchanged, so the evidence is a policy-probability shift rather than a realized-tool flip.

The hidden channel is structurally present in both task splits (contributing 16,384 bits to $\varepsilon_{\text{state}}^{\text{UB}}$) but becomes action-relevant only on planning tasks. The read-only proxy variant underperforms due to an observational bottleneck; analysis is deferred to the supplemental material.

²Doing so would require controlled variation of hidden-state structure under fixed observable marginals.

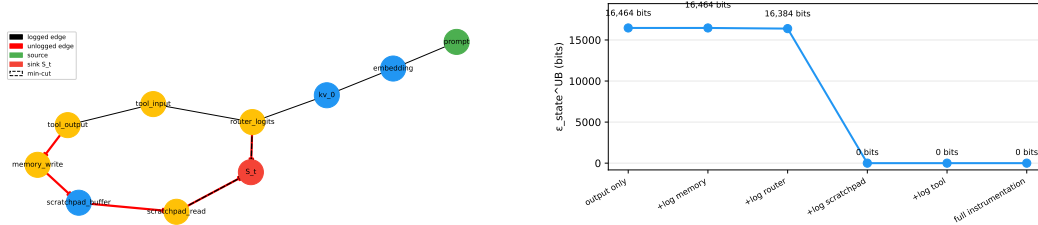


Figure 1: Left: Time-unrolled DAG extracted from Qwen2.5-7B-Instruct ReAct execution (logical edges omitted for clarity). Red edges are unlogged; dashed edges form the min-cut. Right: Logging ablation curve. Full logging removes all hidden capacity, driving $\varepsilon_{\text{state}}^{\text{UB}}$ to zero.

Task	Condition	JS (bits; nats)	95% CI (bits)
<i>Intervention</i>			
Calculator (dormant)	mask 0.7	0.0000; 0.0000	[0, 0]
Planning (active)	mask 0.7	0.0215; 0.0149	[0.0094, 0.0410]
Calculator (dormant)	replace 1.0	0.0000; 0.0000	[0, 0]
Planning (active)	replace 1.0	0.0055; 0.0038	[0.0016, 0.0131]
<i>Controlled replay</i>			
Calculator (dormant)	scratchpad removed	0.0000; 0.0000	[0, 0]
Planning (active)	scratchpad removed	0.0163; 0.0113	[0.0124, 0.0208]

Table 3: Dynamic certificates under the same unlogged-scratchpad topology. Dormant calculator tasks show no detectable scratchpad influence ($\delta_{\text{act}}^{\text{LB}} \approx 0$); active planning tasks show positive action-distribution shift. The headline replay contrast reveals the hidden channel matters for planning but not calculation.

273 6.3 Mid-Spectrum: LLaDA Denoising Trajectory

274 LLaDA uses the same dual-certificate framework, but the activation profile is indexed over time rather
 275 than module. The hidden denoising channel exists throughout the K -step trajectory, while decision
 276 relevance concentrates at the final binding step. We use LLaDA-8B-Instruct with $K=10$ denoising
 277 steps on tool-selection prompts. A Gaussian perturbation ($\sigma=5.0$) is applied to layer-1 activations at
 278 steps {2, 4, 6, 8, 10}; a late-layer control perturbation tracks broad sensitivity. Each point uses $n=20$
 279 trajectories and a trajectory-block bootstrap with 1,000 resamples.

280 The pattern is late-binding (Fig. 2): steps 2–8 remain near the floor (0.001–0.006 bits), while step 10
 281 rises to $\delta_{\text{act}}^{\text{LB}} = 0.110$ bits (95% CI [0.052, 0.234]). The control layer also rises at step 10 (0.092
 282 bits, CI [0.043, 0.186]), so the sensitivity is global. LLaDA carries hidden capacity throughout
 283 denoising; it becomes action-relevant only at the discrete tool-choice binding step, placing it mid-
 284 spectrum between the ReAct near-zero and multi-agent near-saturation witnesses. Although the
 285 perturbation is injected into continuous hidden activations, the reported $\delta_{\text{act}}^{\text{LB}}$ is computed over the
 286 final discrete tool-token action distribution; the InfoNCE/MINE branch is relevant only when A_t
 287 itself is continuous.

288 6.4 Near-Saturation: Multi-Agent Private-Report Intervention

289 The multi-agent peer-report channel (Table 2) gives a near-saturation witness: the hidden communica-
 290 tion edge carries nearly the full binary-action ceiling. The controller delegates evidence gathering to
 291 a worker, so the worker’s private report becomes an epistemic edge in the decision graph. The worker
 292 sees a benchmark item and writes a short report; the controller sees only that report and chooses one
 293 API from {calculator, search}. The task pool combines 200 GSM8K arithmetic items [Cobbe
 294 et al., 2021] and 200 HotpotQA multi-hop fact questions [Yang et al., 2018] with seed 42.

295 The main intervention replaces the worker report with a report from the opposite task class while
 296 holding the controller protocol fixed. A neutral replay removes the report content, and a same-class
 297 shuffle replaces it with another report from the same task class. An oracle-tag condition directly names
 298 and flips the API label to measure the binary ceiling. For each task and condition, the controller is

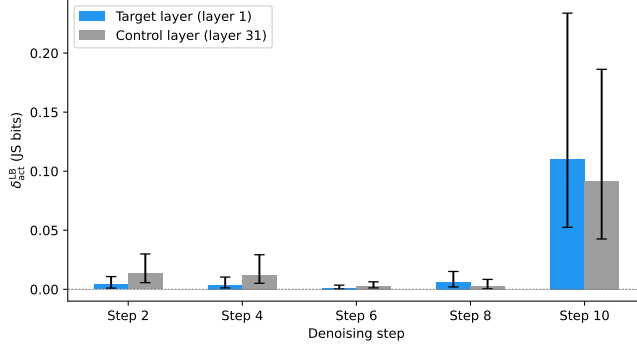


Figure 2: Temporal activation profile for LLaDA-8B-Instruct. Early denoising steps have low dynamic certificates; the final action-forming step rises to $\delta_{act}^{LB} = 0.110$ bits for both target and control layers, indicating that decision relevance is concentrated at the final binding step.

Condition	δ_{act}^{LB} (JS bits)	95% CI (bits)
Neutral replay	0.358	[0.324, 0.392]
Counterfactual evidence	0.901	[0.873, 0.928]
Same-class shuffle	0.084	[0.060, 0.109]
Oracle tag upper-bound	1.000	[1.000, 1.000]

Table 4: Dynamic certificates for the multi-agent deployment. Counterfactual evidence nearly saturates the binary action ceiling (0.901 bits out of 1.0), while same-class shuffling (0.084 bits) and neutral replay (0.358 bits) remain much smaller. Only the dynamic certificate reveals the hidden channel’s behavioral impact.

sampled $k=8$ times; labels are parsed as {calculator, search, invalid}; and $JS(P_{base}, P_{condition})$ is averaged with a task-block bootstrap.

Reclassifying the worker from Trusted to Untrusted shifts audit strategy from detection to mandatory topology redesign (see §5.1 for the trust taxonomy); the dynamic certificate $\delta_{act}^{LB} = 0.901$ bits is unchanged by this reclassification.

7 Discussion

The two-evidence-types frame. Existing audit frameworks often collapse hiddenness into a scalar. The results here separate two kinds of evidence—topology and behavior—plus a trust axis that determines action. This decomposition makes the gap closable rather than merely diagnosable.

Limitations. The fixed-deployment-marginal scope of Definition 2 applies; distribution shift under red-teaming or deployment drift is not covered. The structural-access closer (Proposition 1) requires software orthogonality for tightness of the per-edge additive decomposition; shared hidden state across unlogged channels weakens the min-cut interpretation without invalidating the upper bound (Appendix A). The gray-box-access closer (Proposition 2) requires the probe to satisfy the conditional Markov chain $X_t \rightarrow S_t \rightarrow A_t$ given $\tilde{T}_t$; probe misspecification can underestimate δ_{act} .

Trust boundaries in practice. Trust sensitivity (§5.1) applies directly: misclassifying an Untrusted component as Trusted underestimates ε_{state}^{UB} , and the default-deny principle recommends treating components without cryptographic or organizational trust anchors as Untrusted.

Future Directions. The main formal next step is lifting the finite-support assumption to continuous state spaces. Empirically, sharper error analyses for InfoNCE/MINE-style continuous-action certificates would strengthen the dynamic certificate, and active probe selection could replace hand-designed probes. On the engineering side, connecting the formal proofs (Appendix D) to a verified audit toolchain would yield machine-checked certificates per deployment. Natural Language Autoencoders (NLAs) [Fraser-Taliente et al., 2026] could supply readable proxy probes for Proposition 2, but only

323 when their generated descriptions satisfy the same conditional-Markov requirement; failures would
324 instantiate the probe-misspecification limitation above.

Acknowledgments

We thank the developers of Qwen2.5 and LLaDA for the open-weight models used in the ReAct and diffusion-LM experiments; the creators and maintainers of GSM8K and HotpotQA for benchmark data used in the multi-agent setting; and the Hugging Face Transformers, PyTorch, NumPy, scikit-learn, NetworkX, Matplotlib, Lean 4, Lake, and Mathlib communities for the software infrastructure supporting the experiments, figures, and formalization. Computations ran on a single Apple M4 Max workstation.

References

- Rudolf Ahlswede, Ning Cai, Shuo-Yen Robert Li, and Raymond W. Yeung. Network information flow. *IEEE Transactions on Information Theory*, 46(4):1204–1216, 2000. doi: 10.1109/18.850663.
- Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeshwar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and Devon Hjelm. Mutual information neural estimation. In *Proceedings of the 35th International Conference on Machine Learning*, pages 531–540, 2018.
- Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022. doi: 10.48550/arXiv.2212.03827.
- Stephen Casper, Carson Ezell, Charlotte Siegmann, Noam Kolt, Taylor L. Curtis, Benjamin Bucknall, Andreas Haupt, Kevin Wei, Jérémy Scheurer, Marius Hobbhahn, et al. Black-box access is insufficient for rigorous AI audits. In *The 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024. doi: 10.1145/3630106.3659037.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. doi: 10.48550/arXiv.2110.14168.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.
- Abbas El Gamal and Young-Han Kim. *Network Information Theory*. Cambridge University Press, 2011.
- Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral consequences of information removal from a representation. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021.
- Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C. Bogdan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, Julius Tarnig, Alex Wu, Jeff Wu, Yang Zhang, Daniel M. Ziegler, Evan Hubinger, Joshua Batson, Jack Lindsey, Samuel Zimmerman, and Samuel Marks. Natural language autoencoders produce unsupervised explanations of LLM activations. *Transformer Circuits Thread*, 2026. URL <https://transformer-circuits.pub/2026/nla/index.html>.
- Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 9574–9586, 2021.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025. doi: 10.48550/arXiv.2502.09992.
- Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

- 372 Shauli Ravfogel, Yanai Elazar, Hila Gonen, Michael Twiton, and Yoav Goldberg. Null it out:
373 Guarding protected attributes by iterative nullspace projection. In *Proceedings of the 58th Annual*
374 *Meeting of the Association for Computational Linguistics*, pages 7237–7256, 2020.
- 375 Alexander Matt Turner, Lisa Thiergart, David Udell, Gavin Leech, Ulisse Mini, and Monte Mac-
376 Diarmid. Activation addition: Steering language models without optimization. *arXiv preprint*
377 *arXiv:2308.10248*, 2023.
- 378 Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive
379 coding. *arXiv preprint arXiv:1807.03748*, 2018.
- 380 Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov,
381 and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question
382 answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language*
383 *Processing*, pages 2369–2380, 2018. doi: 10.18653/v1/D18-1259.
- 384 Huiling Zhen, Weizhe Lin, Renxi Liu, Kai Han, Yiming Li, Yuchuan Tian, Hanting Chen, Xiaoguang
385 Li, Xiaosong Li, Chen Chen, Xianzhi Yu, Mingxuan Yuan, Youliang Yan, Peifeng Qin, Jun Wang,
386 Yu Wang, Dacheng Tao, and Yunhe Wang. DLLM agent: See farther, run faster. *arXiv preprint*
387 *arXiv:2602.07451*, 2026. doi: 10.48550/arXiv.2602.07451.
- 388 Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan,
389 Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J.
390 Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson,
391 J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to AI
392 transparency. *arXiv preprint arXiv:2310.01405*, 2023.

393 **NeurIPS Paper Checklist**

- 394 **1. Claims** Do the main claims made in the abstract and introduction accurately reflect the paper’s
395 contributions and scope?
- 396 *Answer:* Yes. All claims are matched by theoretical derivations (§4–5) and empirical evidence (§6).
- 397 **2. Limitations** Does the paper discuss limitations of the work?
- 398 *Answer:* Yes. Limitations are discussed in §7, including the fixed-marginal scope, software-
399 orthogonality dependence, and probe-misspecification risk.
- 400 **3. Theory Assumptions and Proofs** Did you state the full set of assumptions of all theoretical
401 results, and did you include complete proofs of all theoretical results?
- 402 *Answer:* Yes. All assumptions are stated with each result; proofs appear in the main text and
403 Appendix A, with mechanically verified proofs provided in Appendix D.
- 404 **4. Experimental Result Reproducibility** Did you provide a way to reproduce the experimental
405 results?
- 406 *Answer:* Yes. Detailed protocols are in Appendix B; code, data, and formal proofs are in the
407 supplementary material.
- 408 **5. Open access to data and code** Did you include the code, data, and instructions needed to
409 reproduce the main experimental results?
- 410 *Answer:* Yes. Code, precomputed results, and reproduction instructions are in the supplementary
411 material.
- 412 **6. Experimental Setting/Details** Did you specify all the training details (e.g., data splits, hyperpa-
413 rameters, how they were chosen)?
- 414 *Answer:* Yes. All hyperparameters, data splits, and bootstrap settings are reported in §6 and Ap-
415 pendix B.
- 416 **7. Experiment Statistical Significance** Does the paper report error bars suitably and correctly
417 defined or other appropriate information about the statistical significance of the experiments?

418 *Answer:* Yes. All tables report 95% trajectory- or task-block bootstrap CIs; the method is described
419 in Appendix B.

420 **8. Experiments Compute Resource** For each experiment, does the paper provide sufficient
421 information on the computer resources (type of compute workers, memory, time of execution) needed
422 to reproduce the experiments?

423 *Answer:* Yes. All pipelines ran on a single Apple M4 Max workstation (§6).

424 **9. Code Of Ethics** Have you read the NeurIPS Code of Ethics and ensured that your research
425 conforms to it?

426 *Answer:* Yes. The work focuses on safety-oriented auditing and does not involve human subjects,
427 privacy-sensitive data, or dual-use risks beyond those discussed.

428 **10. Broader Impacts** If appropriate for the scope and focus of your paper, did you discuss potential
429 negative societal impacts of your work?

430 *Answer:* Yes. Potential dual-use concerns are acknowledged in §7; the framework is inherently
431 defensive in scope.

432 **11. Safeguards** Do you have safeguards in place for responsible release of models with a high risk
433 for misuse (e.g., pretrained language models)?

434 *Answer:* N/A. No new models are released; existing open-weight models are used as audit targets
435 without modification.

436 **12. Licenses** Did you cite the creators and respect the license and terms of use of existing assets?

437 *Answer:* Yes. All used assets are publicly available and cited where applicable.

438 **13. New Assets** Are you releasing new assets (datasets, code, models), and did you document them
439 properly?

440 *Answer:* No. Code, data, and formal proofs will be released upon acceptance.

441 **14. Crowdsourcing and Research with Human Subjects** Did you use crowdsourcing or conduct
442 research with human subjects?

443 *Answer:* N/A. No human subjects were involved.

444 **15. IRB Approvals** Did you describe potential participant risks and obtain Institutional Review
445 Board (IRB) approvals (or an equivalent approval/review)?

446 *Answer:* N/A. IRB approval is not applicable.

447 **16. Declaration of LLM usage** Does the paper describe the usage of LLMs if it is an important,
448 original, or non-standard component of the core methods in this research?

449 *Answer:* N/A. The LLM (Qwen2.5) is only the audit target, not a component of the auditing method.

A Network Information Theory Background for Proposition 1

Conditional channel capacity. For each unlogged edge e in the time-unrolled DAG G_t , the conditional capacity c_e is the maximum information that can flow through e under the worst-case distribution consistent with the system protocol. It is a structural upper bound from interface specifications: token window length, hidden-dimension size, quantization level, or discrete-state cardinality. A text channel with budget K tokens over vocabulary $\mathcal{V}$ has $c_e \leq K \log |\mathcal{V}|$; a vector state of dimension d at quantization Q has $c_e \leq d \log Q$; a discrete scheduler with $|\Omega|$ states has $c_e \leq \log |\Omega|$. The certificate remains valid under any tighter $c_e^* \geq I(X_e; Y_e | \tilde{T}_t)$, so auditors who can measure the empirical input distribution to a channel may substitute a tighter bound.

Software orthogonality and additive decomposition. A set of edges is *software-orthogonal* if, conditioned on $\tilde{T}_t$, the channel outputs factor: $p(\{y_e\}_{e \in E'} | \{x_e\}, \tilde{T}_t) = \prod_{e \in E'} p(y_e | x_e, \tilde{T}_t)$.

Lemma 1 (Additive decomposition). *Under software orthogonality, the induced cut capacity decomposes additively: $C_{\text{cut}}(\Omega) \leq \sum_{e \in E(\Omega, \Omega^c)} c_e^{\text{budget}(e)}$, where $\text{budget}(e) = \text{proto for Trusted edges and adv for Untrusted edges}$.*

Cut-set upper bound and additive decomposition. Let $G_t = (V_t, E_t)$ be the time-unrolled DAG. For a cut $\Omega \subseteq V_t$ separating source nodes from a target, the *induced cut capacity* $C_{\text{cut}}(\Omega)$ is the supremum over joint input distributions $p(X_\Omega | \tilde{T}_t)$ of the directed conditional mutual information $I(X_\Omega \rightarrow Y_{\Omega^c} | \tilde{T}_t, X_{\Omega^c})$. The cut-set upper bound [El Gamal and Kim, 2011] states that any information flowing from sources in Ω to a target in Ω^c is bounded above by $C_{\text{cut}}(\Omega)$; minimizing over cuts gives the tightest such bound. Proposition 1 applies this bound to the unrecorded trace M_t . We use only the upper-bound direction; the network-coding achievability results [Ahlswede et al., 2000] are not invoked. Under software orthogonality, Lemma 1 shows $C_{\text{cut}}(\Omega) \leq \sum_{e \in E(\Omega, \Omega^c)} c_e$, reducing the certificate to a discrete min-cut problem on G_t (the form used in experiments). When software orthogonality fails for E' , the per-edge decomposition does not hold, but the joint cut capacity is bounded by the direct-product alphabet: $C_{\text{cut}}(\Omega) \leq \log(\prod_{e \in E'} |\mathcal{X}_e|)$ via $H(\{X_e\}) \leq \sum_e \log |\mathcal{X}_e|$. This joint-alphabet bound is looser than the per-edge sum under orthogonality but guarantees a finite upper bound regardless of cross-channel interference.

Proof sketch of Proposition 1. The argument has two steps. (1) Trace-gap identity: let $M_t := T_t \setminus \tilde{T}_t$; then $H(S_t | \tilde{T}_t) = H(S_t | T_t) + I(S_t; M_t | \tilde{T}_t)$ by the chain rule, where $H(S_t | T_t) = \varepsilon_{\text{nominal}}$. (2) Cut-set bound: M_t is d -separated from S_t by the cross-cut signals $(X_\Omega \rightarrow Y_{\Omega^c}, X_{\Omega^c})$ conditional on $\tilde{T}_t$, so $I(S_t; M_t | \tilde{T}_t) \leq C_{\text{cut}}(\Omega)$ for every cut $\Omega \in \text{Cuts}(U_t \rightarrow S_t)$. Minimizing over cuts and combining with step (1) yields the proposition. Corollary 1 follows by substituting Lemma 1. The full derivation is mechanized in the formal proofs (Appendix D).

B Experimental Protocols

B.1 Static Certificate Computation Protocol

Inputs: deployment architecture specification, logging manifest, and protocol budgets (token window sizes, hidden dimensions, quantization levels). Procedure: (1) construct the time-unrolled DAG G_t ; (2) partition edges into logged and unlogged; (3) verify software orthogonality (Lemma 1) or add shared-state edges explicitly until orthogonality holds; (4) assign capacity c_e per unlogged edge (§5.1); (5) enumerate reachable cut sets $\mathcal{C}_{\text{unlogged}}$ via min-cut after collapsing logged edges to zero residual capacity; (6) compute $\varepsilon_{\text{state}}^{\text{UB}} = \varepsilon_{\text{nominal}} + \min_C \sum_{e \in C} c_e$ (Corollary 1). For the logging ablation, repeat (2)–(6) for the ordered manifests in §6.1. See `experiments/7.1_static_certificate/README.md` for implementation.

B.2 Dynamic Certificate Estimation Protocols

- Proxy certificate.** Run N trajectories recording $(\tilde{T}_t, Z_t^{(k)}, A_t)$; train cross-fitted predictors $p_\theta(A_t | \tilde{T}_t)$ and $p_\phi(A_t | \tilde{T}_t, Z_t^{(k)})$; compute $\hat{I}_k = L_{\text{trace}} - L_{\text{trace}+Z^{(k)}}$; estimate trajectory-block bootstrap CIs. Repeat for random, label-permuted, and PCA-projected probes.

- 497 2. **Controlled replay certificate.** For each $\tilde{T}_t$, run the same system under wild and controlled
 498 replayed missing-state conditions; estimate $\hat{\Delta} = \hat{JS}(P_{\text{wild}}, P_{\text{replay}} \mid \tilde{T}_t)$; aggregate over trace
 499 slices. Valid only under the replay condition in §5.2.
- 500 3. **Intervention certificate.** Inject ξ_{hidden} at perturbation levels $\{0, \sigma_1, \dots\}$ while holding $\tilde{T}_t$
 501 fixed; estimate $\hat{I}(\xi_{\text{hidden}}; A_t \mid \tilde{T}_t)$ via conditional action-distribution shift (discrete actions)
 502 or InfoNCE/MINE (continuous). Repeat on dormant and active tasks under the same
 503 topology.
- 504 4. **Multi-agent private-report certificate.** Sample $N/2$ arithmetic and $N/2$ factual-retrieval
 505 tasks; generate private evidence reports under instructions forbidding direct API names;
 506 sample the controller action distribution k times under wild, neutral replay, opposite-
 507 class report, same-class shuffled report, and oracle-tag conditions; parse actions into
 508 $\{\text{calculator}, \text{search}, \text{invalid}\}$; compute per-task JS divergence with task-block boot-
 509 strap CIs. The main contrast is the opposite-class worker-report intervention.

510 See the README files for experiments 7.2, 7.3, and 7.6 for implementation details.

511 C Synthetic Ground-Truth Validation

512 To test whether the certificates recover known hidden influence, we construct a synthetic agent with a
 513 binary hidden variable $H \sim \text{Bern}(0.5)$ injecting influence β_h into action logits; the unlogged-channel
 514 capacity $c_H = 1$ bit is known. We run both the static min-cut ($\varepsilon_{\text{state}}^{\text{UB}} = 1$ bit) and the proxy CE-diff
 515 estimator on 1,000 trajectories at five influence levels.

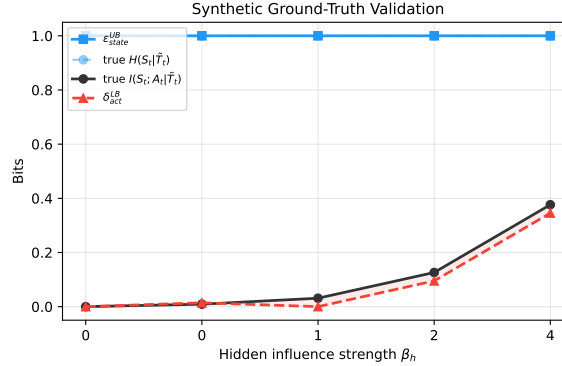


Figure 3: Synthetic ground-truth. $\varepsilon_{\text{state}}^{\text{UB}}$ (blue) equals the true $H(S_t \mid \tilde{T}_t) = 1$ bit. $\delta_{\text{act}}^{\text{LB}}$ (red) tracks true $I(S_t; A_t \mid \tilde{T}_t)$ from below, with negative values at low β_h reflecting estimator variance near the noise floor.

516 Fig. 3 shows $\varepsilon_{\text{state}}^{\text{UB}} = 1$ bit matches the true hidden entropy (tight when no redundant unlogged capacity
 517 exists). The dynamic bound $\delta_{\text{act}}^{\text{LB}}$ tracks the true conditional MI from below at all $\beta_h \geq 1$. Unlike the
 518 Qwen deployment where a near-deterministic baseline suppresses the proxy signal ($H(A_t \mid \tilde{T}_t) \approx 0$),
 519 the synthetic environment injects controlled stochasticity, so the proxy certificate can become tight
 520 when structural capacity is behaviorally active and the observational ceiling permits detection.

521 D Formalization of Information-Theoretic Claims

522 The artifact mechanizes the main finite-discrete information-theoretic claims over
 523 FinitePMF, using Mathlib finite sums and real logarithms. The theory core is built in
 524 FiniteQuerySandbox.InfoTheory:

525 **Foundational objects.** A FinitePMF is a Lean structure with exactly three fields: a mass function
 526 pmf of type $\alpha \rightarrow \mathbb{R}$, a proof pmf_nonneg of $\forall x, 0 \leq \text{pmf}(x)$, and a proof sum_one of $\sum_x \text{pmf}(x) =$
 527 1. Every theorem quantifies over finite index types through the Lean typeclass assumptions Fintype
 528 and DecidableEq, so marginalization, entropy, KL, and conditional mutual information are all

explicit `Finset.univ` sums. No measure-space, sigma-algebra, countable-additivity, or continuity axiom is imported for probability.

Real logarithms and zero mass. The artifact uses Mathlib’s `Real.log` on $\mathbb{R}$ rather than postulating a separate logarithm. Base-2 quantities are defined by division by `Real.log 2`, and the positivity/nonzero facts for `Real.log 2` are proved locally from Mathlib theorems. Mathlib makes `Real.log 0 = 0`; the entropy summand `negMulLog2 p` is therefore a total real-valued function with the zero-mass term evaluating to zero. Proofs that require logarithm algebra case-split on zero mass and establish strict positivity before using identities such as `Real.log_div`. Thus divisions by zero and support conditions are handled inside the Lean statements, not hidden as informal side assumptions.

Entropy. `negMulLog2` and `entropyOf/entropy` define Shannon entropy via $-p \log_2 p$ over finite types. `negMulLog2_nonneg` and `entropy_nonneg` establish $-p \log_2 p \geq 0$ for $p \in [0, 1]$; `entropy_le_log_card` gives $H(P) \leq \log_2 |\alpha|$ via KL nonnegativity.

KL divergence and DPI. `kl_nonneg` proves $\sum p_x \log(p_x/q_x) \geq 0$ via $\log x \leq x - 1$ with case-splitting on zero mass. `condMarkov` defines $X \rightarrow Y \rightarrow Z \mid W$ via PMF factorization, and `cond_dpi` proves the conditional DPI $I(X; Z \mid W) \leq I(Y; Z \mid W)$ via KL nonnegativity, chain-rule identities, and tuple reindexing. `condEntropy_nonneg` and `mutualInfo_nonneg` prove non-negativity of conditional entropy $H(X \mid Y) \geq 0$ and mutual information $I(X; Y) \geq 0$ over finite types.

Type equivalence. `FinitePMF.comapEquiv`, `entropyOf_equiv_eq`, and `entropyOf_mul_log2` establish entropy invariance under type isomorphism and the $\cdot \ln 2$ conversion.

Supporting modules: `IdentifiabilityGap` (Theorem 1), `DualCertificate` (Propositions 1–2 plus the finite-cardinality corollary `prop1_static_ub_bounded`), `QuantizedBound` (discrete-to-analog bridge $H(P) \leq d \log_2 Q$), `TraceRecoverability` and `TraceRecoverabilityBridge` (Corollary 2 surface), `QuotientFactorization` (semantic-closure factorization surface), the impossibility family `FiniteQueryDecisionImpossibility`, `PredictabilityRouteImpossibility`, `SeparatedPackingImpossibility`, and `GeometricTools/CoveringBound/PACBounds/Tools` (geometry and PAC utilities). The top-level `FiniteQuerySandbox` root imports all modules for single-pass verification; `FiniteQueryAudit` is an alternate root alias. Legacy module names are retained for compatibility.

Main-Text Item	Lean File	Lean Theorem	Status
Thm 1 (Identifiability Gap)	<code>IdentifiabilityGap</code>	<code>identifiability_gap_extremes</code>	C
Cor 2 (Autoregressive zero-cut)	<code>TraceRecoverability</code>	<code>no_eis_autoregressive</code>	C
Prop 1 (Structural-access closer)	<code>DualCertificate</code>	<code>prop1_static_ub</code>	C/P
Static cardinality corollary	<code>DualCertificate</code>	<code>prop1_static_ub_bounded</code>	C
Prop 2 (Gray-box-access closer)	<code>DualCertificate</code>	<code>prop2_dynamic_lb</code>	C
Quantized Entropy Bound	<code>QuantizedBound</code>	<code>quantized_vector_entropy_bound</code>	C

Table 5: All results mechanized from finite-discrete definitions. **C** = fully mechanized (no external premises). **C/P** = structural reduction: the cut-set capacity bound is taken as an external premise (proved on paper in Appendix A); the formal theorem derives the certificate from it.